Data Science

Lecture 04: Machine Learning

Prof. Dr. Michael Granitzer

Speaker notes

Converted from LNKB-tex part-en-ds-chp4-machine-learning.tex. Builds on Lecture 02 (probability, expectation, sampling and the standard error) and Lecture 03 (Bayes for a decision, and conditional independence, which naive Bayes pushes to its extreme). Maximum likelihood is introduced in Module 4, before logistic regression uses it.

Two sessions. Session 1 covers Modules 1–3 (the learning problem, inductive bias, linear regression with LMS); session 2 covers Modules 4–6 (classification, naive Bayes, measuring performance). The Module 4 slide marks the boundary.

Where We Left Off

In Lectures 02–03 *we* wrote the model down: the probabilities, the graph. From today the machine chooses the model from data — and it can only do that because we restrict what it may choose.

Write the model
Lectures 02–03

→

Learn it from data
Modules 1–5 — a form, a loss, an optimiser

→

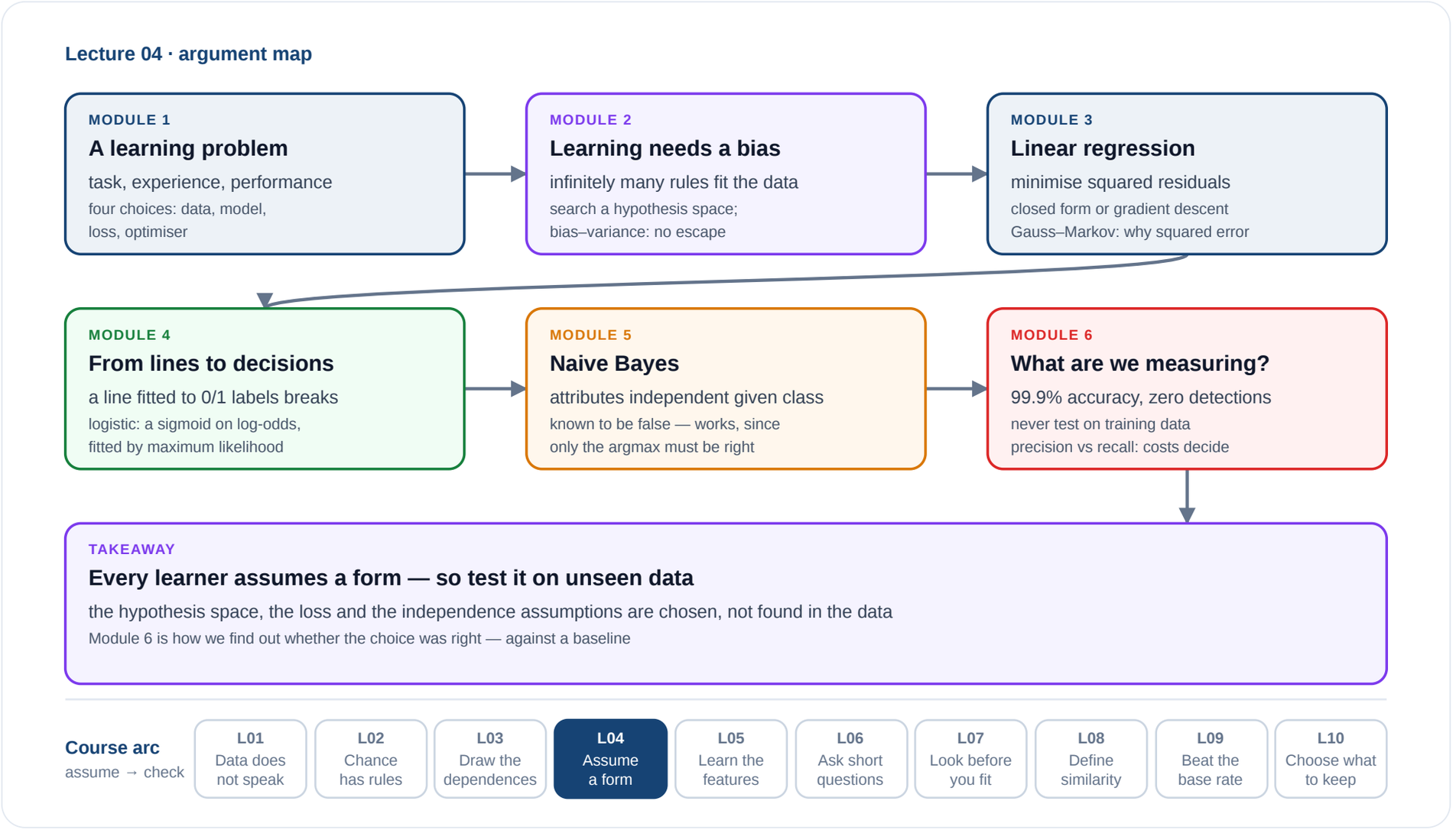
Check it on unseen data
Module 6

The base-rate trap of Lecture 02 returns in Module 6: a model with 99.9% accuracy that never detects a single case.

Speaker notes

Module 2 (inductive bias) is the hinge of the lecture and the arc's name for it: every model after it is "a form someone chose". Say that out loud when logistic regression and naive Bayes arrive. The lecture spans two sessions; the Module 4 slide marks the boundary.

Argument Map



Speaker notes

What a learning problem is, and why it cannot be solved without a bias: infinitely many rules fit the data. Then three forms, each an assumption — a line, with squared error justified by Gauss–Markov; a sigmoid on log-odds, fitted by maximum likelihood; attributes independent given the class (naive Bayes), which is false and works because only the argmax has to be right. Module 6 is the check, and it has its own trap. The bottom row is the lecture in one sentence.

Module 1

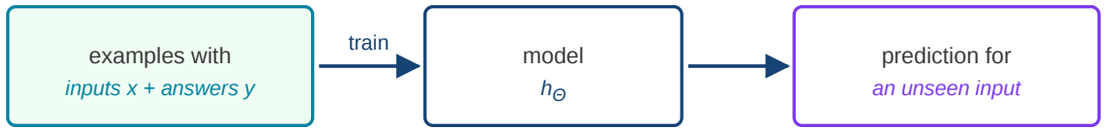
What Is a Learning Problem?

> ***Guiding Question:*** *What has to be true of a task — and of the data — before "learn it from examples" is a sensible answer?*

Learning From Answers

A radiologist has labelled ten thousand scans. A bank has ten years of loans and knows which ones defaulted. In both cases **every example carries its answer**.

Supervised: every training example carries the answer



The learner is told the correct output for each case it sees.

What information does the learner receive during training?

Speaker notes

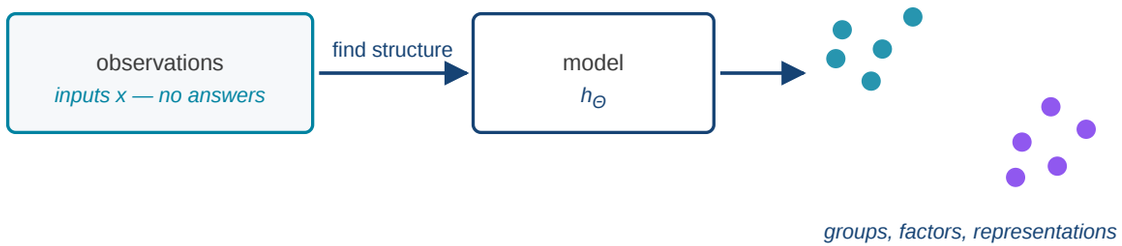
Ask before the click: what exactly does the learner see? The answer — an input *and* the output we wanted — is what makes this the easiest setting, and the most expensive one, because somebody had to produce every label.

Good examples to name: image classification, medical diagnosis, house prices. The last one previews regression: the "answer" need not be a class.

Learning Without Answers

Now the same bank wants to know **what kinds of customer it has**. Nobody has ever written down the list — the categories are what we are looking for.

Unsupervised: no answers — only the observations themselves



Structure has to be found, not reproduced.

What can we learn when nobody tells us the correct answer?

Speaker notes

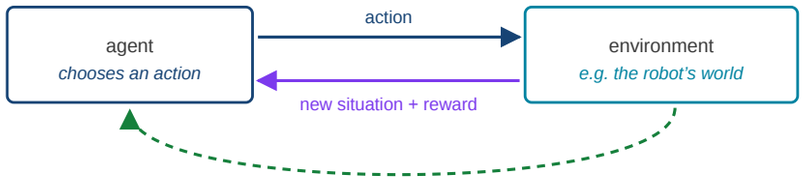
Let the room answer first: groups of similar customers, the few dimensions that carry most of the variation, what is unusual. Name clustering, dimensionality reduction, latent structure — but only after they have said it.

The honest difficulty to flag: without an answer key there is no unambiguous notion of "correct", so evaluation becomes a genuine problem. That returns in the clustering lecture.

Learning by Acting

A robot has to cross a room. Nobody can label the correct motor command for every situation it might find itself in — and it only discovers whether a move was good **after making it**.

Reinforcement: no answer is given — only the consequence of acting



the consequence changes what the agent does next

The learner must act to find out whether the action was good.

How can a learner improve when it must act in order to find out whether the action was any good?

Speaker notes

Action and consequence are already drawn; click 1 closes the loop back into the agent's behaviour, click 2 states that the learner must act to find out. Only now name the vocabulary: **agent**, **environment**, **action**, **reward**.

The point that distinguishes this from supervised learning: the feedback says how good the outcome was, not what the right action would have been — and it may arrive long after the action that caused it.

Same Goal, Different Information

Same goal, different information

	WHAT IT IS GIVEN	WHAT FEEDBACK IT GETS
Supervised	inputs + answers	the answer, every time
Unsupervised	inputs only	nothing
Reinforcement	the situation it acts in	a reward, after acting

Speaker notes

Reveal one row per click, so the comparison is assembled from the three examples the students have just seen rather than presented as a taxonomy.

Ask which paradigm a spam filter uses (supervised), a recommender trained on clicks (arguably all three), a game engine (reinforcement). The boundaries are softer than the table suggests, and saying so is more honest than a clean three-way split.

Same Goal, Different Information

Same goal, different information

	WHAT IT IS GIVEN	WHAT FEEDBACK IT GETS
Supervised	inputs + answers	the answer, every time
Unsupervised	inputs only	nothing
Reinforcement	the situation it acts in	a reward, after acting

> The paradigms differ in **what the learner is given** and **what feedback it gets** — not in what we ultimately want from it.

What Is Common to All Three?

Nobody wrote the rule

The mapping from situation to answer is **too complex or too implicit** to specify by hand.

Examples exist

There is data — or experience — in which the desired behaviour is demonstrated, even if it was never described.

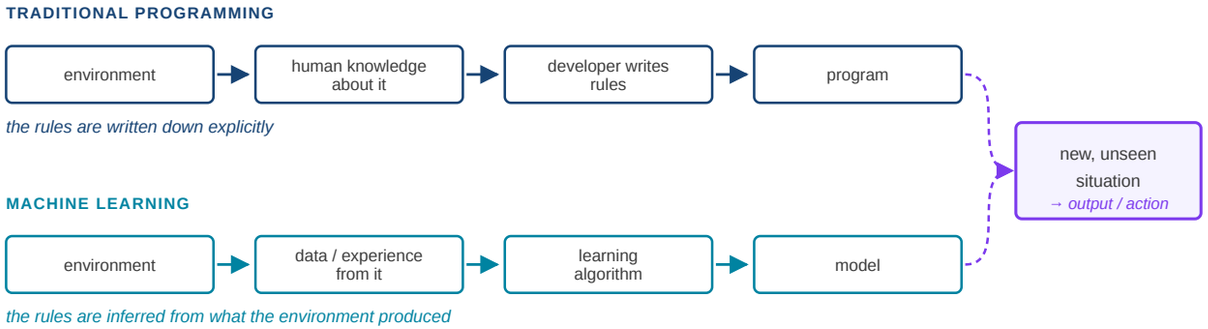
So what replaces the rule that nobody wrote?

Speaker notes

This is the bridge to the next slide. Hold the question: in all three paradigms, something is derived from data that a developer would otherwise have had to write down. That "something" is the model — and comparing the two ways of getting it is the next slide.

Two Ways to Get Behaviour

Both are judged on situations neither of them has seen



Speaker notes

Three clicks. First the programming pipeline: the developer holds knowledge about the environment and writes it down as rules. Second the learning pipeline: the environment produces data, and an algorithm derives the model. Third — the point of the slide — **both arrive at the same destination**: a situation neither the developer nor the learner has seen.

Ask between clicks two and three: what is the same in both rows? The environment on the left, and the demand on the right.

What Actually Changed

TRADITIONAL PROGRAMMING

The developer's **knowledge becomes rules**. Deduction: rules plus input give the output.

The assumptions are visible — they are in the code.

MACHINE LEARNING

The environment's **data becomes a model**. Induction: examples plus a learning procedure give a rule.

The assumptions are still there — but now they are in the data, the model family and the objective.

The rules do not disappear when nobody writes them. They become **harder to inspect** — which is why the rest of this lecture is about making every part of that pipeline explicit.

Speaker notes

Deliberate callback to the three modes of reasoning: programming is deductive, learning is inductive. Students who saw that module will recognise the shape, and the guarantee that came with each.

The warning is the honest framing of the trade and sets up the design questions at the end of the module.

What Would It Mean for a Machine to Learn?

Before any definition — three questions you would have to answer about *any* system before you could say it had learned anything:

- ? What is it supposed to get better at?
- ? What does it get to look at while getting better?
- ? How would we know whether it improved?

Speaker notes

Questions first, names second (P4). Let the room produce the three questions in their own words — they always do — and only then attach Mitchell's (1997) vocabulary. A definition that arrives as a label for something already understood costs nothing to remember.

What Would It Mean for a Machine to Learn?

Before any definition — three questions you would have to answer about *any* system before you could say it had learned anything:

- ? What is it supposed to get better at?
- ? What does it get to look at while getting better?
- ? How would we know whether it improved?

Task T

what the system should learn to do

Experience E

what it learns from

Performance P

how we decide it improved

Chess

♟

A program that plays, loses, and plays again — millions of times against itself.

- → **Task** — play chess well
- → **Experience** — games played against itself, and positions from expert play
- → **Performance** — fraction of games won; rating against other engines

Speaker notes

The classic example, and worth one observation: the experience here is *generated by the system itself*. No one labelled the good moves — the outcome of the game did.

A Large Language Model

¶

A model shown enormous quantities of text, asked only to continue it.

- → **Task** — predict the next word
- → **Experience** — a very large corpus of text, then human feedback
- → **Performance** — likelihood of held-out text; later, human judgement

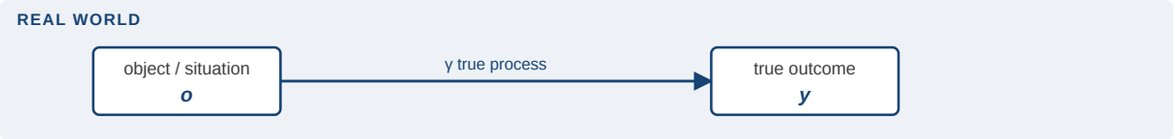
Two systems with nothing in common — a board game and a text corpus — fit the **same three-part description**. That is what makes it a definition rather than a description of one program.

Speaker notes

Pause on the task: "predict the next word" looks far too weak to produce what these systems do. That is exactly why the choice of a *proxy* task is a real design decision — it must force the model to represent what actually matters.

What Would We Ideally Like to Know?

A process in the world turns situations into outcomes — we cannot see it directly



There is some real process that turns a situation into an outcome. Why can we not simply write it down?

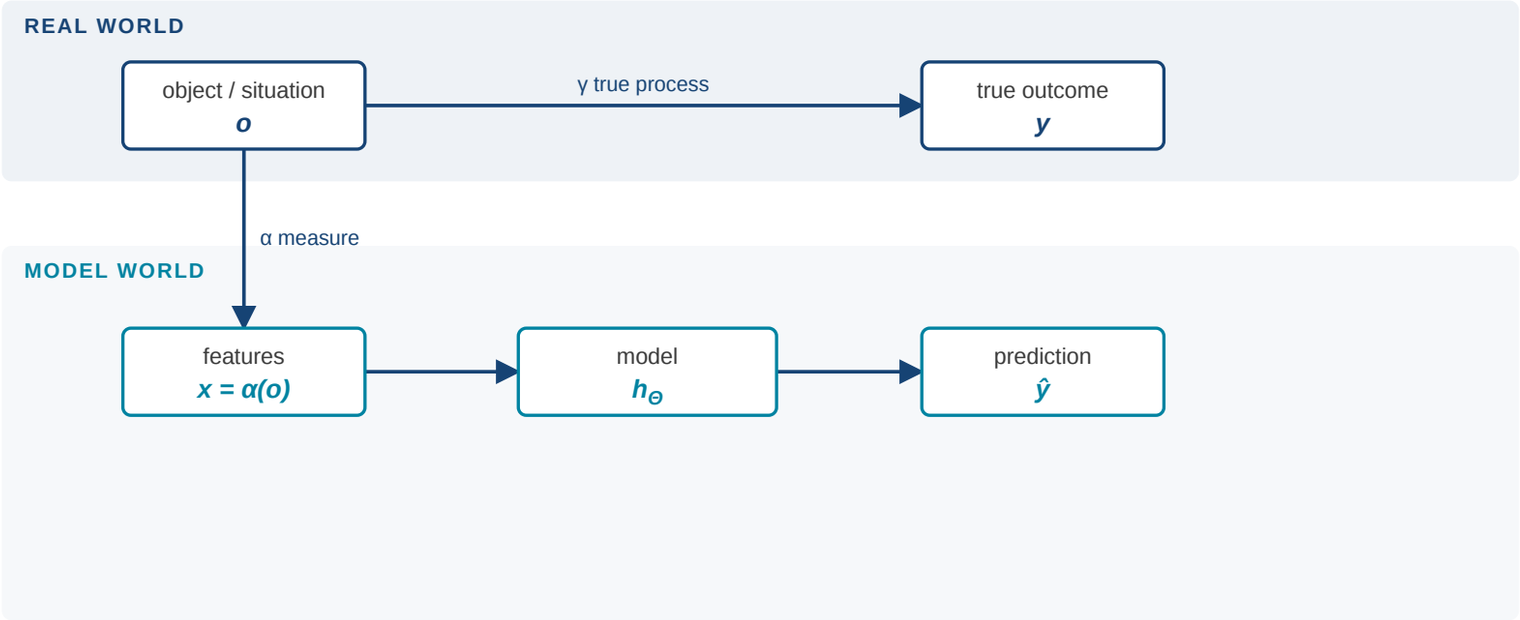
Speaker notes

Only the real-world band is drawn: an object or situation o, the true process γ, the outcome y. Let the room answer the question — the process is unknown, or too complex, or not accessible to us at all. That inaccessibility is the reason everything that follows exists.

Notation, kept from the source: γ **is the ideal target function** on real-world objects.

From the Real World to a Model of It

The model maps features to a prediction



Two Worlds, One Correspondence

$$\gamma(o) \quad \rightsquigarrow \quad h_\Theta\big(\alpha(o)\big) \quad \equiv \quad h_\Theta(x)$$

γ

the **real process** — unknown, and not ours to write down

α

the **representation** — what we chose to measure

h_Θ

the **model** — our approximation, with parameters Θ

We never learn the world. We build a **model world** and approximate the process inside it.

Speaker notes

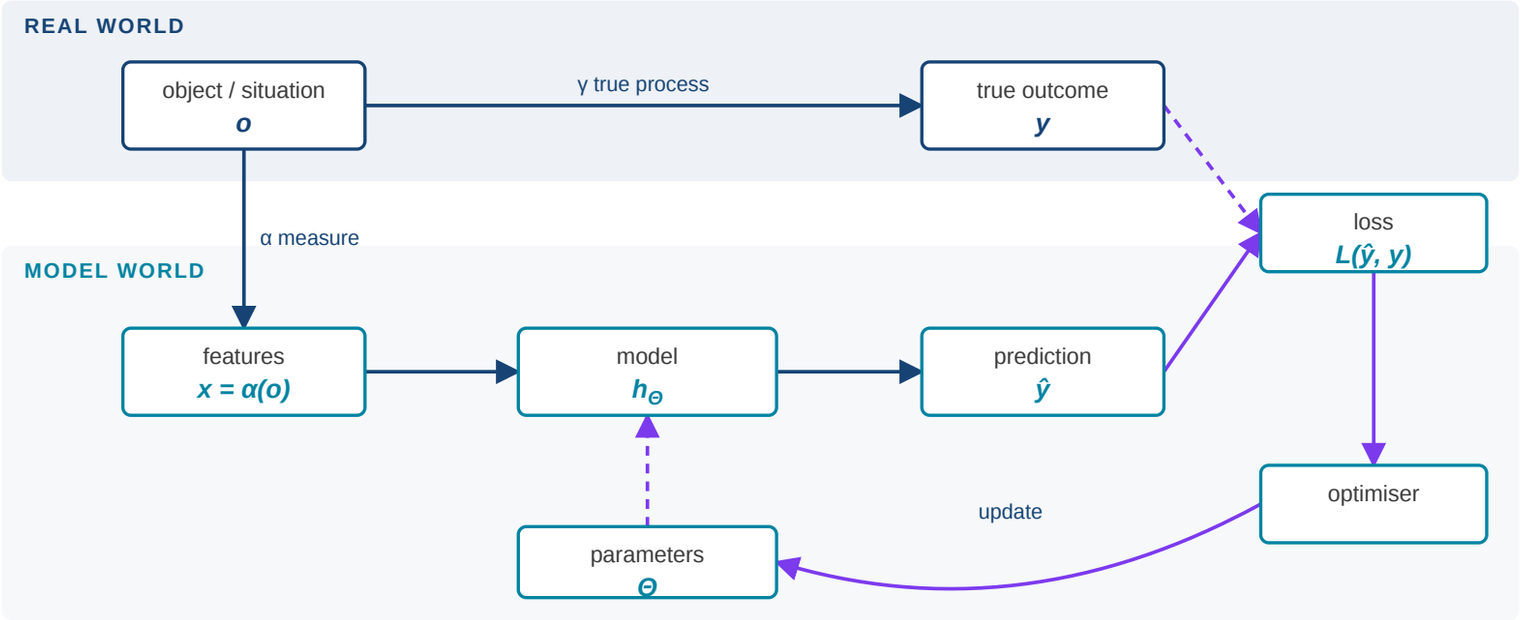
Two clicks: first the measurement — the **model-formation function** α turns an object into features $x = \alpha(o)$ — then the model h_Θ and its prediction.

Be precise about the notation, because it is easy to garble: γ is the real process, α **is the representation**, h_Θ is the model and Θ are its parameters. α is *not* the parameters.

The relation on the fragment is the heart of the module: two worlds, and a correspondence between them that is only ever approximate. Most practical failures are failures of α — the features did not carry what γ depends on.

Closing the Loop

The optimiser changes the parameters — and the training loop closes



Speaker notes

Two more clicks complete the figure. Click 1 adds the **loss**, which is the first thing on the slide that compares the model world with the real one: it needs $\hat{y}$ *and* the outcome y we actually wanted. Click 2 adds the **optimiser**, which changes the **parameters** Θ — and the loop closes.

Read it aloud once as a sentence: measure the situation, predict, compare with what happened, adjust, repeat. Every method in this course fills these boxes differently.

The Four Choices

Data
what we measure

→

Model
h_Θ — what it may express

→

Loss
what counts as wrong

→

Optimiser
how Θ is searched

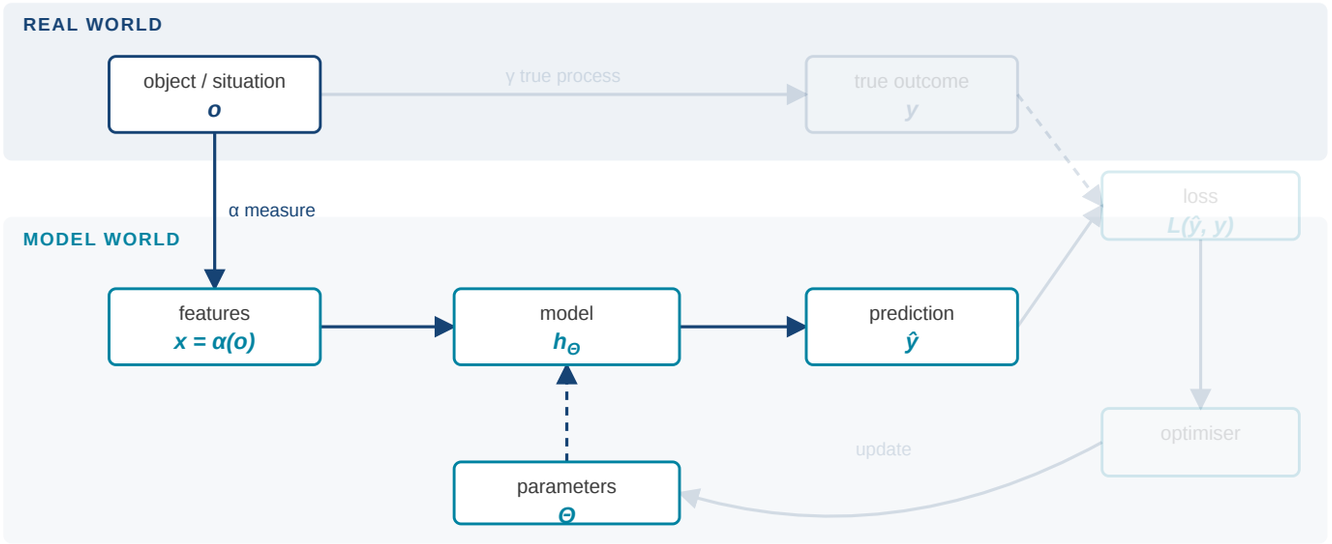
The optimiser has settings of its own that are not learned from the data — **hyperparameters**. Choosing them is part of the outer process, not of the loop.

Speaker notes

The compact restatement of the figure, and the map for the whole course: every algorithm later is a different set of these four. Have students place linear regression into the four boxes before moving on.

Training Is Not Using

Inference: a new, unseen situation follows one path — the training machinery is gone



the learned parameters Θ are now fixed

Which of these boxes still exist when the system is actually deployed?

Speaker notes

One click dims everything the training loop needed. What remains is a single path: a new situation, measured, through the model, to a prediction — with Θ now fixed.

Ask the question before clicking. Students routinely assume the loss is somehow still "checking" the model in production; it is not, because in production nobody tells us y. That gap — no answers after deployment — is what monitoring and the evaluation methodology have to work around.

The Task: Predicting Acceleration

The world

O — vehicles

$Y = \mathbb{R}$ — time to accelerate to 100 km/h

$\gamma(o)$ — the true acceleration time of vehicle o

The model world

α — measure wheels, horsepower, weight, torque

X — those four numbers, for every vehicle

h_Θ — a linear function of them

Speaker notes

Deliberately mundane. The value is that every abstract symbol of the learning system — γ the real process, α the representation (what we measure), h the model — now has a concrete referent students can argue with, especially α, where "which four numbers" is a real modelling decision.

The Data

wheels	horsepower	weight	torque	acceleration time
4	150	1350	250	9.8
4	320	1600	430	4.9
6	95	2100	210	16.2
…	…	…	…	…

The last column is y; the rest is $\mathbf{x}$.

Speaker notes

Ask which column is likely useless (wheels, in a passenger-car dataset) and what would happen if it were included anyway — an invitation to regularisation later.

The Remaining Two Pieces

Loss function

Mean squared error — the average of the squared differences between predicted and observed acceleration times.

Learning algorithm

LMS — least mean squares, which adjusts the parameters to reduce that error.

A learning problem becomes concrete the moment you can name its **task, experience and performance measure**, and its system becomes buildable the moment you can name its four choices: **data, model, loss and optimiser**.

Speaker notes

Close by returning to the four-box map. This unit is the template students should apply when reading any paper: identify the four boxes before judging the result.

Designing the Learning Problem

Before any model is chosen, four things have to be decided — and each of them is a claim about the world. First, what the system is for:

Decide	Ask
The task	What should the system predict or decide? What is one unit of prediction? What counts as success?
The target	What is γ, in words a domain expert would accept? Is the label we have the thing we care about, or a proxy for it?

> A proxy label is the most consequential choice here: "was diagnosed" is not "had the disease", "was clicked" is not "was good".

Speaker notes

The proxy question is the one with the most consequences in practice: "did the patient have a disease" versus "was the patient diagnosed", "is this content good" versus "was it clicked".

Designing the Learning Problem — Data

THE DATA

- ? Which observations represent the problem?
- ? How were the examples sampled?
- ? Will future cases look like these?

THE REPRESENTATION A

- ? What are the inputs, concretely?
- ? Which features are available — and which are merely convenient?
- ? What is lost in the measurement?

Speaker notes

Every box is a modelling decision that the mathematics will later take for granted. The sampling question returns in evaluation: a model is only tested on the population its examples were drawn from.

Designing the System

MODEL AND OBJECTIVE

- ? What does the model family assume, and how flexible is it?
- ? Does minimising this loss correspond to what we actually want?

LEARNING

- ? Which procedure searches the parameters?
- ? How sensitive is the result to initialisation and randomness?

Designing the System — Evaluation

EVALUATION AND GENERALISATION

- ? Which metric matters, and which data is held out?
- ? How uncertain is the estimate of performance?
- ? What happens when the environment changes?

UNCERTAINTY AND USE

- ? Which uncertainty must be represented explicitly?
- ? Is the prediction informing a decision — and what do the errors cost?

Speaker notes

Close the module by pointing at the questions rather than at topics — the rest of this lecture and the lectures after it then read as answers to them. The last box is deliberately the one that matters most: a prediction exists to support a decision, and its errors have costs.

Designing the System — Evaluation

EVALUATION AND GENERALISATION

- ? Which metric matters, and which data is held out?
- ? How uncertain is the estimate of performance?
- ? What happens when the environment changes?

UNCERTAINTY AND USE

- ? Which uncertainty must be represented explicitly?
- ? Is the prediction informing a decision — and what do the errors cost?

These are the questions the rest of the course answers. Each lecture takes one or two of them and makes them precise.

Takeaway

> Before any model is fitted, the design questions are answered — what one prediction is, which data stands for the world, what counts as success. Each answer is an **assumption**; together they decide whether any model can succeed.

Speaker notes

Module 1 closes here (course-local slide). The module's units each end on their own point; this slide names what they share, in the words of the course arc: the design questions are the first assumptions of every learning project, made before the inductive bias of Module 2 enters.

Module 2

How Learning Is Possible

> ***Guiding Question:*** *Infinitely many rules fit the data we have seen. What licenses a machine to prefer one of them?*

Learning as Search

The **hypothesis space** $\mathcal{H}$ is the set of all functions the model family can express — every possible value of the parameters Θ. Learning is the search for the best $h_\Theta \in \mathcal{H}$.

Exhaustive search

Examine all of $\mathcal{H}$. Feasible only when the space is small (with a few special exceptions).

Heuristic search

Use assumptions to explore only part of the space. Efficient, but the optimum may be missed.

Except for very simple problems you cannot have both: searching **all** hypotheses **efficiently and robustly**.

Speaker notes

Framing learning as search makes the later algorithms comparable: gradient descent, tree induction and k-means are all heuristic searches through differently shaped spaces.

Inductive Reasoning and Inductive Bias

Inductive reasoning: generalising from seen examples to unseen ones — from observations to a rule that predicts future observations.
Inductive bias: the assumptions by which an algorithm generalises. The choice of hypothesis space is already one; so are assumptions about noise, outliers, the data distribution and the shape of errors.

Without an inductive bias, a system **cannot generalise** at all.

Speaker notes

The claim in the box is stronger than students expect and worth defending: with no assumptions, any behaviour on unseen data is consistent with the training data, so nothing can be inferred. Bias is not a defect to be removed, it is the precondition for learning — the question is only whether it is the right bias.

The Bias–Variance Trade-Off

Learning trades **fitting the observed data** against **holding to prior assumptions about unseen data**.

Bias

Systematic error: averaged over many training sets, the model still misses the truth, because its assumptions rule it out — the *stubborn learner*.
High bias → robust to noise, but misses real structure → **underfitting**.

Variance

How much the learned model changes from one training set to the next — it follows irregularities of the particular sample.
High variance → captures patterns, but also noise → **overfitting**.

More parameters generally means more variance. The trade-off is not a defect of particular algorithms — it is a property of inductive inference.

Speaker notes

Do not give the decomposition formula here; the intuition carries the chapter. Ask which of the two a very deep decision tree suffers from, and which a constant predictor has — the extremes make the axis concrete before any model does.

Takeaway

> Learning is a **search** through a hypothesis space, steered by an **inductive bias** you chose — knowingly or not. Every model sits somewhere on the bias–variance axis, and no model escapes it.

Module 3

Linear Regression

> ***Guiding Question:*** *Why squared error — and what exactly does "linear" forbid?*

Classification versus Regression

Classification

Y is a finite set of labels.

Will this customer default — yes or no?

Regression

$Y = \mathbb{R}$, the target is a quantity.

What credit line should this customer receive?

The learning machinery is the same; only the shape of Y — and therefore the loss — changes.

Speaker notes

The distinction is fixed by the target set Y, not by the inputs: the same customer record supports a classification task (default, yes or no) and a regression task (credit line in euros). The usual losses follow from Y: 0/1 error or cross-entropy for a finite Y, squared or absolute error for $Y = \mathbb{R}$. Ordinal targets (ratings 1–5) and counts lie in between; treating them as unordered classes ignores their order, treating them as real numbers ignores their discreteness.

The Model

Linear regression predicts the target as a weighted sum of the features:

$$h_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + \ldots + w_p x_p = \mathbf{w}^\top \mathbf{x}$$

with $x_0 = 1$ absorbing the intercept.

The hypothesis space is $\mathbb{R}^{p+1}$: one hypothesis per weight vector.

Speaker notes

Absorbing the intercept means prepending a constant feature $x_0 = 1$ to every input, so that w_0 is handled like any other weight and the design matrix $\mathbf{X}$ has a column of ones. The hypothesis space $\{h_{\mathbf{w}} : \mathbf{w} \in \mathbb{R}^{p+1}\}$ is continuous and finite-dimensional, so the best hypothesis is found by calculus — setting the gradient of the objective to zero — rather than by enumeration.

A weight w_j is the change of the prediction per unit change of x_j with all other features held fixed. Its size depends on the unit of x_j: weights of features on different scales are not directly comparable, and a large weight does not by itself mean an important feature.

The Objective: Residual Sum of Squares

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}\in\mathbb{R}^{p+1}} \mathrm{RSS}(\mathbf{w}), \qquad \mathrm{RSS}(\mathbf{w}) = \sum_{i=1}^{n} \left(y_i - \mathbf{w}^\top \mathbf{x}_i\right)^2$$

RSS measures **effectiveness** — how good the predictions are. Do not confuse it with **efficiency**, which is about runtime and computational effort.

Speaker notes

RSS/n is the mean squared error and $\sqrt{\mathrm{RSS}/n}$ the root-mean-square error, which is in the unit of the target. Squaring makes the objective differentiable everywhere and convex in $\mathbf{w}$ — a quadratic with a unique minimum when $\mathbf{X}^\top\mathbf{X}$ is invertible — and weights large residuals disproportionately: a residual twice as large costs four times as much, so a single outlier can move the fit considerably.

Effectiveness and efficiency are independent: a model can be accurate and slow to fit, or fast and inaccurate. Calling an accurate model "efficient" is a common misuse of the term.

One Feature: The Closed Form

For a single feature the minimiser can be written down directly:

Slope

$$\hat{w}_1 = \frac{\sum_i (x_i - \overline{x})(y_i - \overline{y})}{\sum_i (x_i - \overline{x})^2}$$

The covariance of x and y over the variance of x.

Intercept

$$\hat{w}_0 = \overline{y} - \hat{w}_1 \overline{x}$$

The fitted line passes through the point of means.

Speaker notes

Derivation. Setting $\partial \mathrm{RSS}/\partial w_0 = -2\sum_i (y_i - w_0 - w_1 x_i) = 0$ gives $w_0 = \overline{y} - w_1 \overline{x}$. Substituting this into $\partial \mathrm{RSS}/\partial w_1 = -2\sum_i x_i (y_i - w_0 - w_1 x_i) = 0$ gives the slope formula. Dividing numerator and denominator by $n - 1$ shows $\hat{w}_1 = \mathrm{cov}(x, y)/\mathrm{var}(x) = r_{xy}\, s_y / s_x$, with r_{xy} the correlation coefficient; the slope is undefined when all x_i are equal.

Consequences: the residuals sum to zero, and the line passes through $(\overline{x}, \overline{y})$. Regressing x on y gives a different line, because least squares measures errors vertically, in y only; the two lines coincide only when $|r_{xy}| = 1$.

How Much Variance Is Explained?

$$\mathrm{TSS} = \mathrm{ESS} + \mathrm{RSS}, \qquad R^2 = \frac{\mathrm{ESS}}{\mathrm{TSS}} = 1 - \frac{\mathrm{RSS}}{\mathrm{TSS}}$$

where TSS is the total, ESS the explained and RSS the residual sum of squares (the split holds for a least-squares fit with an intercept).

R^2 is the fraction of the target's variation the model accounts for: 0 means no better than predicting the mean, 1 means an exact fit.

Training R^2 never decreases when a feature is added, however useless the feature. A high training R^2 is not evidence that a model generalises — test R^2 and adjusted R^2 can and do fall.

Speaker notes

Definitions: $\mathrm{TSS} = \sum_i (y_i - \overline{y})^2$, $\mathrm{ESS} = \sum_i (\hat{y}_i - \overline{y})^2$, $\mathrm{RSS} = \sum_i (y_i - \hat{y}_i)^2$. The split holds because, with an intercept, the residuals sum to zero and are orthogonal to the fitted values, so the cross term vanishes. For a single feature, $R^2 = r_{xy}^2$.

Without an intercept, or on test data, $1 - \mathrm{RSS}/\mathrm{TSS}$ can be negative: the model is then worse than predicting the mean. Training R^2 cannot fall when a feature is added, because the larger model contains the smaller one (set the new weight to 0), so its minimal RSS is at most as large. Adjusted $R^2 = 1 - (1 - R^2)\,\frac{n-1}{n-p-1}$ charges for each of the p features and falls when a feature reduces RSS by less than a random feature would on average.

Higher Dimensions

With p features the same objective in matrix form,

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

gives the optimum in one step whenever $\mathbf{X}^\top \mathbf{X}$ is invertible.

> Invertibility fails when features are linearly dependent or when $p > n$ — more unknowns than equations. Both are common in practice; regularisation is the standard remedy.

Speaker notes

Derivation: $\mathrm{RSS}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$ has gradient $-2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w})$; setting it to zero gives the normal equations $\mathbf{X}^\top\mathbf{X}\,\mathbf{w} = \mathbf{X}^\top\mathbf{y}$, with $\mathbf{X}$ of size $n \times (p+1)$ including the column of ones. $\mathbf{X}^\top\mathbf{X}$ is invertible exactly when the columns of $\mathbf{X}$ are linearly independent, which requires $n \geq p+1$.

When it is singular the normal equations still have solutions, but infinitely many; the pseudo-inverse selects the one of minimal norm. Strongly correlated features leave the matrix invertible but ill-conditioned, and the weights become unstable. Numerical software solves the system by QR or Cholesky decomposition instead of forming the inverse. Ridge regression replaces $\mathbf{X}^\top\mathbf{X}$ by $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$, which is invertible for every $\lambda > 0$. $p > n$ is the normal situation in genomics and text, where features outnumber examples.

Takeaway

Linear regression fits a weighted sum by minimising squared residuals. Its solution is built from **covariance over variance**, and R^2 reports how much variation it explains — not whether it will generalise.

Linear in the Parameters, Not in the Features

A model is **linear** when it is linear in the **weights** $\mathbf{w}$. The features may be arbitrary transformations of the input.

Plain

$$h(\mathbf{x}) = w_0 + w_1 x$$

a straight line

Polynomial

$$h(\mathbf{x}) = w_0 + w_1 x + w_2 x^2$$

a curve — still a *linear model*

Basis functions

$$h(\mathbf{x}) = \sum_j w_j \phi_j(\mathbf{x})$$

any fixed feature map ϕ

What *linear* forbids: a transformation that has to be learned from the data. The feature map is fixed before fitting; only the weights are learned.

Speaker notes

Linearity in the weights is what makes RSS a quadratic function of $\mathbf{w}$ and keeps the normal equations valid: with a feature map ϕ, the design matrix simply has entries $\Phi_{ij} = \phi_j(\mathbf{x}_i)$. Fixed maps include polynomials, interactions $x_1 x_2$, $\log x$, indicator variables for categories, periodic terms $\sin(2\pi t/T)$ with known period T, and radial basis functions with fixed centres.

Not expressible in this way: models whose transformation has parameters fitted to the data, such as $w_1 \sin(w_2 x)$ with a learned frequency w_2, radial basis functions with learned centres, or the hidden layers of a neural network. Such models are non-linear in their parameters; their losses are in general non-convex and have no closed-form minimiser.

Consequences

The good

The closed-form solution and all statistical guarantees carry over unchanged.

The catch

The feature map is a **modelling decision**. Choose too rich a basis and the model overfits; too poor and it underfits.

> Increasing the polynomial degree turns a dial on the bias–variance axis.

Speaker notes

What carries over is everything that holds for a linear model in $\phi(\mathbf{x})$: the closed form, the projection geometry, and the Gauss–Markov guarantee, provided the model is correctly specified in the chosen basis. Choosing the basis is choosing the hypothesis space — the inductive bias stated explicitly.

Too rich a basis has low bias and high variance; with as many basis functions as data points the fit interpolates the training data. Too poor a basis has high bias: no amount of data lets a straight line fit a sine. Raw polynomial features $x, x^2, \ldots, x^k$ are strongly correlated and make $\mathbf{X}^\top\mathbf{X}$ ill-conditioned at high degree; rescaling x to $[-1, 1]$ or using orthogonal polynomials mitigates this.

The Question Nobody Asks

We minimised the *squared* residuals. Why squares — why not absolute deviations, or fourth powers?

Speaker notes

Squared error has several independent justifications. It is differentiable and gives a closed form. Its minimiser is a mean: the constant minimising expected squared deviation is the mean, the one minimising expected absolute deviation is the median, so least squares estimates the conditional mean of y. Under the conditions of the Gauss–Markov theorem it gives the linear unbiased estimator of smallest variance, and under Gaussian noise it is the maximum-likelihood criterion. Absolute deviations give least-absolute-deviation regression, which estimates the conditional median and is less sensitive to outliers; higher even powers weight large residuals even more strongly and make the fit more sensitive to them.

Geometric View

The predictions $\mathbf{Xw}$ live in the **column space** of $\mathbf{X}$ — everything the features can express. The observed $\mathbf{y}$ generally does not.

Least squares picks the point of that space **closest to $\mathbf{y}$ in Euclidean distance**: the orthogonal projection. The residual is perpendicular to every feature.

Consequence

The residual carries no remaining linear information about the features.

Speaker notes

The normal equations $\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\hat{\mathbf{w}}) = \mathbf{0}$ state exactly that the residual vector is orthogonal to every column of $\mathbf{X}$. The fitted values are $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ with the hat matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$, the orthogonal projection onto the column space. Pythagoras on $\mathbf{y} = \hat{\mathbf{y}} + \mathbf{e}$, after centring, gives TSS = ESS + RSS.

Adding a feature enlarges the column space, so the closest point can only get closer and RSS cannot increase. "No remaining linear information" means that the residuals are uncorrelated with each feature in the sample; a non-linear pattern of the residuals against a feature remains possible and indicates a misspecified model.

The Gauss–Markov Theorem

Gauss–Markov (informal). Among all estimators that are

- **linear** in the observed outputs $(y_1, \ldots, y_n)$, and
- **unbiased**,

the least-squares estimator has the **lowest variance**. It is the *Best Linear Unbiased Estimator* — BLUE.

Best

lowest variance among the candidates

Linear

a linear function of the observations

Unbiased

right on average over repeated samples

Speaker notes

Formal statement: for $\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\varepsilon}$ with $\mathbf{X}$ fixed and of full column rank, $\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$ and $\mathrm{Cov}(\boldsymbol{\varepsilon}) = \sigma^2\mathbf{I}$, every linear unbiased estimator $\tilde{\mathbf{w}} = \mathbf{C}\mathbf{y}$ has a covariance exceeding $\mathrm{Cov}(\hat{\mathbf{w}}_{\mathrm{OLS}}) = \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1}$ by a positive semi-definite matrix. Hence every linear combination $\mathbf{c}^\top\mathbf{w}$, including each single weight and each prediction, is estimated with the smallest variance by OLS.

The theorem says nothing about non-linear or biased estimators. Mean squared error is $\mathrm{bias}^2 + \mathrm{variance}$, so a biased estimator can be better: for ridge regression there always exists a $\lambda > 0$ whose weights have a smaller mean squared error than OLS (Hoerl & Kennard, 1970). This does not contradict the theorem, which ranks only unbiased linear estimators.

What This Buys, and What It Costs

The payoff

Minimising RSS yields optimal statistical properties under **mild assumptions** — no normality needed, only a correctly specified linear model, linearly independent features, and zero-mean, uncorrelated errors of constant variance.

The price

Alternatives — absolute error, Huber loss — are **more robust to outliers** but give up the BLUE guarantee.

Squared error is not an arbitrary convenience: it is the loss whose minimiser is the best linear unbiased estimator. Departing from it is legitimate — but it is a trade, and you should be able to name what you traded.

Speaker notes

The assumptions in full: the model is linear in the parameters and correctly specified; the features are linearly independent; the errors have mean zero, equal variance and are uncorrelated. Normality is not required for BLUE; it is required for exact t- and F-tests and intervals in small samples. With unequal variances OLS remains unbiased but is no longer best; weighted least squares with weights $1/\sigma_i^2$ restores optimality.

Robust losses trade efficiency under ideal conditions for stability under outliers. The Huber loss is quadratic for small residuals and linear for large ones; absolute error is linear throughout. Their minimisers are not linear in $\mathbf{y}$, so the theorem does not cover them. The same trade appears in descriptive statistics: the mean minimises squared deviations and is pulled by outliers, the median minimises absolute deviations and is not.

Fitting h by Repeated Correction

LMS (least mean squares)

1. Initialise the weights w.
2. Pick a training example $(\mathbf{x}, y)$ from the training set D.
3. Compute the prediction $h_w(\mathbf{x})$ and the error $e = y - h_w(\mathbf{x})$.
4. Update every weight: $w_i \leftarrow w_i + \eta e x_i$.
5. Repeat 2–4 until the error on D stops decreasing.

Speaker notes

The update is a gradient step on the squared error of a single example. With $e = y - \mathbf{w}^\top \mathbf{x}$ and $E = \frac{1}{2} e^2$, $\partial E / \partial w_i = -e\, x_i$, so $w_i \leftarrow w_i - \eta\, \partial E / \partial w_i = w_i + \eta e x_i$. Each weight moves in the direction that reduces this example's error, in proportion to its feature value. The rule is also called the Widrow–Hoff or delta rule.

Examples are usually visited in random order; one pass over D is an epoch. With a constant η the weights do not settle exactly at the least-squares solution but keep fluctuating around it, because every step follows a single noisy example; a decreasing learning rate or averaging of the iterates makes them converge.

What the Update Rule Encodes

Direction

The sign of e decides whether the prediction goes up or down.

Attribution

The factor x_i gives each feature credit proportional to its value.

Step size

The learning rate η trades speed against stability.

Too large an η oscillates or diverges; too small an η converges slowly. It is a hyperparameter — chosen, not learned.

Speaker notes

Direction: if $e > 0$ the prediction was too low, and every weight with $x_i > 0$ increases. Attribution: a feature with $x_i = 0$ receives no update from that example, and features on large numeric scales receive large updates — one reason features are standardised before gradient-based fitting.

Step size: one update changes this example's prediction by $\eta e \|\mathbf{x}\|^2$, so the new error is $e\,(1 - \eta\|\mathbf{x}\|^2)$. The update overshoots the target when $\eta\|\mathbf{x}\|^2 > 1$ and enlarges the error when $\eta\|\mathbf{x}\|^2 > 2$; normalised LMS divides the step by $\|\mathbf{x}\|^2$ for this reason. The learning rate does not appear in the training loss and cannot be fitted by minimising it; it is chosen by comparing runs, typically on validation data.

Two Ways to the Same Minimum

Direct (closed form)

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Exact, one step. Requires forming and inverting a $(p+1) \times (p+1)$ matrix — costly for large p, impossible when it is singular.

Iterative (gradient descent)

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \mathrm{RSS}(\mathbf{w})$$

Approximate, many steps. Scales to large p and n, works when the direct solution does not exist, and extends to models with no closed form at all.

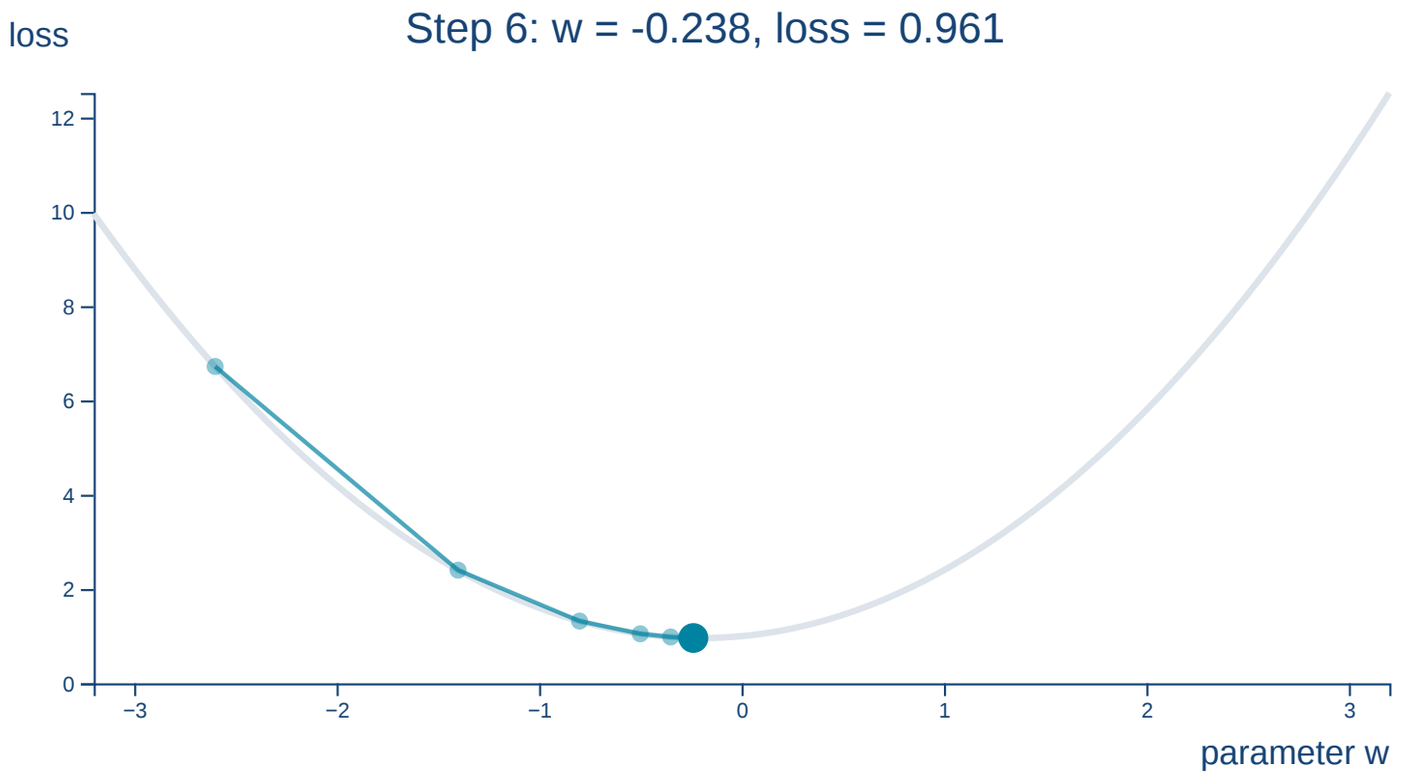
The iterative rule is exactly the **LMS algorithm**: its update $w_i \leftarrow w_i + \eta e x_i$ *is* a gradient step on RSS, taken per example instead of per dataset (the factor 2 is absorbed into η).

Speaker notes

Costs: forming $\mathbf{X}^\top\mathbf{X}$ takes $O(np^2)$ operations and solving the $(p+1)\times(p+1)$ system $O(p^3)$; one full gradient step takes $O(np)$. The gradient is $\nabla\mathrm{RSS}(\mathbf{w}) = -2\mathbf{X}^\top(\mathbf{y}-\mathbf{X}\mathbf{w})$: a full-batch step sums error × feature over all examples, while LMS uses one example per update (stochastic gradient descent).

Because RSS is convex, gradient descent with a small enough learning rate approaches the minimum that the closed form computes. When $\mathbf{X}^\top\mathbf{X}$ is singular the minimum is not unique; started at $\mathbf{w}=0$, gradient descent converges to the minimum-norm solution. Logistic regression and neural networks have no closed-form minimiser, so the iterative method is the general one and the closed form the special case.

Watching Gradient Descent Work

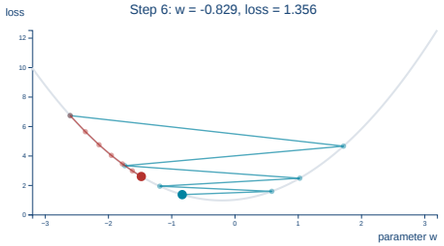


Speaker notes

The curve is a one-parameter stand-in for RSS, $L(w) = w^2 + 0.4w + 1$, with its minimum at $w = -0.2$. A step is $w \leftarrow w - \eta(2w + 0.4)$, so the distance to the minimum is multiplied by $1 - 2\eta$ in every step. With $\eta = 0.25$ the factor is 0.5: starting at $w = -2.6$ (distance 2.4), the distance halves with each step and is 0.0375 after six steps ($w \approx -0.24$).

The steps shrink although η is constant: the step length is $\eta|L'(w)|$, and the gradient is proportional to the distance from the minimum. Steps are therefore largest far from the minimum, where the slope is steepest.

The Learning Rate Decides Everything



Teal: $\eta = 0.9$ — overshoots and oscillates across the minimum. Red: $\eta = 0.05$ — stable, but slow: after six steps it has covered about half the distance.

> The learning rate is a **hyperparameter**: it is chosen, not learned, and both failure modes are silent — one looks unstable, the other looks converged.



Speaker notes

On the same loss the distance to the minimum is multiplied by $1 - 2\eta$ per step. For $\eta = 0.9$ the factor is -0.8: every step jumps across the minimum and removes only 20% of the remaining distance, so after six steps $0.8^6 \approx 26\%$ of it remains. For $\eta = 0.05$ the factor is 0.9, leaving $0.9^6 \approx 53\%$ — the "about half" on the slide. The iteration diverges for $\eta > 1$, where $|1 - 2\eta| > 1$, and $\eta = 0.5$ reaches the minimum in a single step.

In general, gradient descent on a quadratic with Hessian $\mathbf{H}$ is stable only for $\eta < 2/\lambda_{\max}(\mathbf{H})$; here $\mathbf{H} = 2$. The slow run is the harder failure to detect: over a short window its loss curve flattens like that of a converged run, whereas the oscillation of a too-large η is visible in the parameter trace.

Takeaway

> Use the **closed form** when the matrix is small and well-conditioned; use **gradient descent** when it is not — and for most models beyond linear regression, no closed form exists at all.

Module 4

From Regression to Classification

> ***Guiding Question:*** *A decision is a yes or a no. What goes wrong when we fit a straight line to one?*

Speaker notes

Session boundary: the second session starts here. Recall the three-card flow from the opening: Modules 1–3 chose a form and fitted it; Modules 4–5 bring two more forms for decisions; Module 6 checks them on unseen data.

Fitting a Line to Labels

Encode the two classes as $y \in \{-1, +1\}$, fit a linear model, and classify by the sign of the prediction:

$$h(\mathbf{x}) = \operatorname{sign}\left(\mathbf{w}^\top \mathbf{x}\right)$$

One feature

The **discrimination point** x' satisfies

$$w_0 + w_1 x' = 0$$

Everything to one side is one class.

Two features

The **discrimination line** satisfies

$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

In p dimensions it becomes a hyperplane.

Speaker notes

The decision boundary is where the model output crosses zero — a level set. Say it once here; it holds for logistic regression, SVMs and a neural network's final layer alike.

Where It Breaks

Squared loss punishes correctness

A point far on the *correct* side has a large residual, so least squares drags the boundary toward it.

The boundary shifts

Adding well-classified points — or an outlier — moves the discrimination point, even when nothing was misclassified.

The output is also unbounded: $\mathbf{w}^\top \mathbf{x}$ can be 17 or −40, and neither is interpretable as a degree of belief.

Speaker notes

Force the interpretation (P2): ask why "being very right" should ever count as error. Once students see that squared loss is the wrong loss for a decision, the sigmoid arrives as a fix rather than a formula.

Two Problems, One Fix

$\mathrm{sign}(\mathbf{w}^\top \mathbf{x})$ is not differentiable

No gradient means no gradient descent — the optimisation machinery does not apply.

The boundary is unstable

Squared loss on ± 1 labels lets well-classified points shift the discrimination point.

Replace the hard threshold with the **logistic sigmoid**:

$$h_\Theta(\mathbf{x}) = \mathrm{expit}\left(\mathbf{w}^\top \mathbf{x}\right) \approx P(y = 1 \mid \mathbf{x})$$

The output becomes a smooth, bounded quantity that can be read as a probability — and can be differentiated.

Speaker notes

Both problems have the same root: a decision is a step function, and steps have no useful derivative. Softening the step is the standard move; it returns as the activation function in neural networks.

Turning a Model Into an Estimator

We have a model with parameters θ and data D. The model assigns each possible dataset a probability.

The **likelihood** is that probability, read as a function of the **parameters** with the data held fixed:

$$L(\theta) = p(D \mid \theta)$$

The **maximum likelihood estimate** is $\hat{\theta}_{\mathrm{ML}} = \arg\max_\theta p(D \mid \theta)$.

$p(D \mid \theta)$ is a probability over **data**, not over parameters. The likelihood is not a distribution over θ and does not integrate to 1.

Speaker notes

The warning is the single most common confusion in the course. The likelihood is the same expression as the sampling distribution, read along the other axis. A Bayesian treatment puts an actual distribution on θ (a posterior, such as the Beta posterior for a coin); with a flat prior, maximum likelihood keeps only its peak. This is where the two approaches part.

Why the Logarithm

Under the **i.i.d. assumption** — observations independent and identically distributed — the likelihood factorises:

$$L(\theta) = \prod_{i=1}^{n} p(x_i \mid \theta) \qquad \Longrightarrow \qquad \ell(\theta) = \sum_{i=1}^{n} \log p(x_i \mid \theta)$$

Same maximum

$\log$ is monotone, so it does not move the arg max.

Numerically safe

A product of thousands of small probabilities underflows; a sum of logs does not.

Differentiable sums

Sums differentiate term by term — gradient methods apply directly.

> The i.i.d. assumption is doing real work here. Time series, spatial data and grouped data violate it, and the factorisation is then simply wrong.

Example: A Biased Coin

n flips, h heads, parameter $\theta = P(\text{heads})$:

$$\ell(\theta) = h \log \theta + (n - h) \log(1 - \theta) \qquad \frac{d\ell}{d\theta} = \frac{h}{\theta} - \frac{n - h}{1 - \theta} = 0 \qquad \hat{\theta}_{\text{ML}} = \frac{h}{n}$$

The estimator is the observed frequency — the answer intuition would have given. MLE **derives** it rather than assuming it, and the same derivation works where intuition has nothing to offer.

Mini-Exercise

You flip a coin **3 times** and see 3 heads. What is $\hat{\theta}_{\mathrm{ML}}$ — and would you bet that this coin never shows tails?

Speaker notes

Solution on the same slide, click-gated. The three-heads example is the hook for the whole Bayesian thread: students should feel MLE's failure before regularisation and priors are offered as the fix. With a flat prior the posterior is Beta(4, 1): its peak is still 1, but its mean is 0.8.

Mini-Exercise

You flip a coin **3 times** and see 3 heads. What is $\hat{\theta}_{\mathrm{ML}}$ — and would you bet that this coin never shows tails?

$\hat{\theta}_{\mathrm{ML}} = 3/3 = 1$: the estimate rules tails out. Three flips are no basis for that. The estimator uses no **prior** knowledge (coins are usually near-fair) and reports no **uncertainty** — a single point, no error bar. A prior and a posterior close both gaps.

Takeaway

> Maximum likelihood picks the parameters under which the observed data is most probable. It is the bridge from a **model of the world** to a **learning algorithm** — and it is blind to prior knowledge and to its own uncertainty.

Odds and Log-Odds

Odds

The ratio of favourable to remaining cases.

Rolling a six: odds 1 : 5.

Logit

The **log-odds** of a probability p:

$$\operatorname{logit}(p) = \ln \frac{p}{1-p}$$

Range

$p \in (0, 1)$ maps to $(-\infty, +\infty)$ — exactly the range a linear model produces.

The modelling assumption: the logit of the target is a linear combination of the features.

$$\ln \frac{P(y = 1 \mid \mathbf{x})}{1 - P(y = 1 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x}$$

Speaker notes

This is the cleanest way in: we do not force a probability to be linear (it cannot be — it is bounded), we make its log-odds linear. The sigmoid is then not an arbitrary squashing function but the inverse of the logit.

The Model

Inverting the logit gives the **logistic sigmoid**:

$$h_\Theta(\mathbf{x}) = \operatorname{expit}(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

Shape

An S-curve: near-linear around 0, saturating towards 0 and 1 at the extremes.

Reading

$h_\Theta(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$, and the decision boundary is still $\mathbf{w}^\top \mathbf{x} = 0$ — where the sigmoid passes 0.5.

Speaker notes

The boundary is unchanged from linear regression for classification; what changed is the *loss* and the interpretability of the output. Worth stating explicitly, since students often expect a curved boundary.

The Loss: Cross-Entropy

Binary cross-entropy (log loss)

$$L(\Theta) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \ln h_\Theta(\mathbf{x}_i) + (1-y_i)\ln\left(1-h_\Theta(\mathbf{x}_i)\right)\right]$$

For $|Y| > 2$ classes, the multiclass form sums over the classes with a softmax output.

Where does it come from? From the **likelihood**, with the labels now coded as $y_i \in \{0, 1\}$: $P(y_i = 1 \mid \mathbf{x}_i; \Theta) = h_\Theta(\mathbf{x}_i)$ and $P(y_i = 0 \mid \mathbf{x}_i; \Theta) = 1 - h_\Theta(\mathbf{x}_i)$. Maximising the likelihood of the data is minimising this loss.

Speaker notes

The derivation is one line and worth doing aloud: write the likelihood of one example as h^y (1−h)^(1−y), take the product over examples, take the log, negate. Maximum likelihood is the origin of the loss — not a separate idea.

Why It Fixes the Problem

Confident and correct

Loss goes to zero — no penalty for being emphatically right.

Confident and wrong

Loss goes to infinity — exactly the errors we want punished.

There is no closed-form solution; the parameters are found by gradient descent on this loss.

Speaker notes

Contrast directly with the squared-loss failure of regression for classification: cross-entropy punishes confident mistakes and ignores confident successes, which is what a classification loss should do.

Takeaway

Logistic regression is a **linear model of the log-odds**, trained by maximising likelihood. It keeps the linear decision boundary but produces a probability — one whose calibration can be checked — and a loss that punishes the right kind of error.

Module 5

Probabilistic Classification

> ***Guiding Question:*** *The data tells us how attributes behave given the class. We need the reverse. What does the conversion cost?*

Two Directions of the Same Connection

Observed connection

$$A_1, \ldots, A_p \mid B$$

Given the class, how do the attributes behave? — the **generative** direction: modelled class by class.

Reversed connection

$$B \mid A_1, \ldots, A_p$$

Given the attributes, which class? — the diagnosis situation, and the one we actually need.

$$P(B \mid A_1, \ldots, A_p) = \frac{P(B) \cdot P(A_1, \ldots, A_p \mid B)}{P(A_1, \ldots, A_p)}$$

Speaker notes

This is the base-rate lesson turned into an algorithm. A labelled table supports both directions: the discriminative route (logistic regression) models class given attributes directly; the generative route models attributes given class, one class at a time, and converts with Bayes — keeping the class prior explicit. Ask what the generative route buys: the prior can be changed without refitting, and the per-class model can be factorised.

What the Classifier Needs

Class priors

$$P(\text{class} = y)$$

How common is each class, before seeing any attribute?

Attribute likelihoods

$$P(\text{attribute} = v \mid \text{class} = y)$$

How does each class distribute its attribute values?

Given both, the posterior $P(\text{class} \mid \text{attributes})$ follows from Bayes — and the prediction is the class with the highest posterior.

> Estimating $P(A_1, \ldots, A_p \mid B)$ requires the **joint** distribution of all attributes within each class: exponentially many parameters, and never enough data.

Speaker notes

End the unit on the obstacle rather than the solution — the warning is the parameter explosion of a joint distribution, and naive Bayes is the crudest possible independence assumption applied to it.

Generative and Discriminative Classifiers

Generative

Model the **joint** distribution $P(A, B)$ — how the data was produced — then derive $P(B \mid A)$.

Naive Bayes.

Discriminative

Model $P(B \mid A)$ **directly** — only the boundary between classes.

Logistic regression.

Speaker notes

The distinction runs through the rest of machine learning. A generative model can be sampled from and can handle missing attributes; a discriminative model spends all its capacity on the decision. Neither is universally better.

The Naive Assumption

Assume the attributes are **conditionally independent given the class**:

$$P(A_1, \ldots, A_p \mid B) = \prod_{j=1}^{p} P(A_j \mid B)$$

The full joint needs $2^p - 1$ parameters per class for p binary attributes; under the assumption it needs one distribution per attribute — p numbers per class.

The assumption is almost always false. Naive Bayes nonetheless classifies well, because ranking the classes correctly is easier than estimating their probabilities correctly.

Speaker notes

This is conditional independence used as a modelling assumption, taken to its extreme — drawn as a graph, a star with the class at the centre. The note is the honest part: the probabilities are miscalibrated, the argmax often is not.

The Classifier

$$P(B_i \mid A_1, \ldots, A_p) = \frac{P(B_i) \cdot \prod_{j=1}^{p} P(A_j \mid B_i)}{\sum_{l=1}^{k} P(B_l) \cdot \prod_{j=1}^{p} P(A_j \mid B_l)}$$

Score each class

prior × product of attribute likelihoods

Normalise

divide by the total over all classes — the theorem of total probability

Predict

the class with the largest posterior

For prediction alone the denominator can be skipped — it is the same for every class. It is needed only when the probability itself is reported.

Speaker notes

The three-step structure is the whole algorithm. Point out that the denominator is total probability, which the students proved in the probability lecture — nothing new is being introduced here, only assembled.

Example: Enjoy Sport?

Outlook	Temperature	Humidity	Wind	EnjoySport
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rain	mild	high	weak	yes
rain	cool	normal	weak	yes
rain	cool	normal	strong	no

Estimate the priors and the per-attribute likelihoods by **counting**, then score a new day: rain, hot, high, weak.



Speaker notes

Have the group compute P(yes) = 3/6 and P(rain | yes) = 2/3 aloud — the estimation is pure counting, which is what makes the method so cheap. Then score the day before revealing the fragment: the factors are P(rain), P(hot), P(high), P(weak) given each class, times the prior. Exact values: 2/27 for yes, 2/81 for no, so P(yes | day) = 3/4. Then ask what happens for a day that is overcast: P(overcast | no) = 0, which kills the entire product.

Example: Enjoy Sport?

Outlook	Temperature	Humidity	Wind	EnjoySport
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rain	mild	high	weak	yes
rain	cool	normal	weak	yes
rain	cool	normal	strong	no

Estimate the priors and the per-attribute likelihoods by **counting**, then score a new day: rain, hot, high, weak.

$$\text{yes: } \tfrac{1}{2} \cdot \tfrac{2}{3} \cdot \tfrac{1}{3} \cdot \tfrac{2}{3} \cdot 1 \approx 0.074 \qquad \text{no: } \tfrac{1}{2} \cdot \tfrac{1}{3} \cdot \tfrac{2}{3} \cdot \tfrac{2}{3} \cdot \tfrac{1}{3} \approx 0.025$$

$P(\text{yes} \mid \text{day}) = 0.074/(0.074 + 0.025) = 0.75$ — predict **yes**.

The Zero-Frequency Problem

If an attribute value never occurs with a class in the training data, its estimated likelihood is **zero** — and one zero factor drives the whole product to zero, whatever the other attributes say.

Laplace smoothing adds a pseudo-count to every combination, so that unseen does not mean impossible:

$$\hat{P}(A_j = v \mid B) = \frac{n(v, B) + 1}{n(B) + k_j}$$

with k_j the number of values of attribute A_j.

One missing count

vetoes every other piece of evidence

Naive Bayes buys tractability with a **conditional independence assumption it knows to be false** — and works anyway, because classification needs the right argmax, not the right probability.

Speaker notes

With the formula, P(overcast | no) = (0 + 1)/(3 + 3) = 1/6 instead of 0. Close on the epistemic point (P3): a knowingly wrong assumption can still yield a useful model, provided you know which conclusions it does and does not support. Calibrated probabilities are exactly what it does not support.

Module 6

Measuring Performance

> ***Guiding Question:*** *A model reports 99.9% accuracy and has never detected a single case. What should we have measured instead?*

What We Want to Know

> The **true misclassification rate** of a hypothesis h is the probability that it errs on a randomly drawn example $(X, Y) \sim p$:
>
> $$\mathrm{err}^*(h) = P\big(h(X) \neq Y\big)$$

It is defined over the **whole distribution of examples** — which we cannot observe. Since the same $\mathbf{x}$ can come with different labels, even the best possible h errs sometimes: that floor is the **Bayes error**.

Speaker notes

Deliberately parallel to the population/sample distinction from the statistics module: err* is a population quantity. Students who see the analogy will not be surprised by cross-validation. The label is a random variable, not a function of x: two identical-looking cases can end differently, so no amount of data pushes err* below the Bayes error.

What We Can Measure

We estimate $\mathrm{err}^*(h)$ by the error on a set of examples $D_s \subseteq D$ whose true labels are known:

$$\mathrm{err}(h, D_s) = \frac{1}{|D_s|} \sum_{(\mathbf{x},y)\in D_s} 1\big[h(\mathbf{x}) \neq y\big]$$

> The estimate is only as good as the sample. Which examples land in D_s — and whether h has seen them before — decides whether the number means anything.

Speaker notes

Hold-out estimation follows from that warning. Ask what happens if D_s is the data the model was fitted on; the answer motivates hold-out estimation without needing to be told.

Training Error Is Not an Honest Estimate

When the evaluation set equals the training set, $D_{tr} = D$, the resulting number measures how well the model **memorised**, not how well it will generalise.

> Training error is optimistically biased — typically below the true error, often far below. A model with enough capacity can drive it to zero while learning nothing.

Speaker notes

Connect to the bias–variance trade-off: training error falls monotonically with capacity, which is exactly why it cannot be the criterion for choosing capacity.

Four Estimation Schemes

Scheme	How	Trade-off
Hold-out	split D once into a training and a test part; fit on one, evaluate on the other	simple, but wastes data and depends on the split
k-fold cross-validation	partition D into k folds; each fold serves once as the test set; average the k errors	uses all data for both roles; k = 5 or 10 is customary
Leave-one-out	$k = n$: every example is a test set of size one	nearly unbiased, expensive, high variance
Bootstrapping	resample D with replacement to build training sets; evaluate on the examples left out	useful when D is small

Speaker notes

Ask which scheme a Kaggle-style leaderboard implements, and what repeatedly consulting a fixed test set does to its status as an estimate — the test set becomes training data for the modeller.

Misclassification Costs

Not every error costs the same. A missed diagnosis and a false alarm have different consequences, so counting errors equally is itself a modelling assumption.

> A **cost matrix** assigns a cost to each (true class, predicted class) pair; the objective becomes expected cost rather than error rate.

Speaker notes

This is the hinge into precision and recall. Ask the group to set the two costs for a spam filter and then for a cancer screen; the ratios they choose differ by orders of magnitude, and both are defensible.

Takeaway

An error number is meaningless without the sampling scheme that produced it. **Never evaluate on data the model was fitted on**, and state the cost model when errors are not interchangeable.

A Number Alone Means Nothing

"95% accuracy" is not a result until you know what a trivial method achieves.

Trivial classification

Always predict the **majority class**.

Trivial regression

Always predict a **central tendency** — the mean or median of y.

Random guessing

Draw a label from the marginal distribution $p(y)$.

Speaker notes

Insist on the habit: report the baseline next to the model, always. This is the cheapest defence against self-deception in the whole course.

Example: Why Accuracy Deceives

A screening test for a disease affecting **1 in 1,000** people.

The "model"

Always predict *healthy*.

Its accuracy

99.9% — and it has never detected a single case.

> On imbalanced data the majority baseline is nearly unbeatable by accuracy, and completely useless. The metric hides that the model is useless.

Speaker notes

The same base-rate structure as the Bayes unit, now as an evaluation trap. This is the motivation for precision and recall.

Why the Error Rate Is Not Enough

For multiclass classification $h_\Theta : X \to Y$ with $Y \subseteq \{1, \ldots, N\}$, the plain error rate:

Fails on imbalance

The majority baseline scores high while detecting nothing.

Hides the distribution

Which classes the errors fall on stays invisible.

Hides the kind of error

Missing assignments and imprecise assignments count the same.

Speaker notes

Three distinct failures, deliberately separated: imbalance, distribution across classes, and type of error. Each one motivates a different part of what follows.

The Confusion Matrix

Rows are true classes, columns are predictions; cell (i, j) counts the examples of class i that h assigned to j. The diagonal is correct, and every off-diagonal cell names a specific confusion.

> The matrix is the raw material: every measure below is a ratio of its cells. Read it before reading any summary number.

Speaker notes

On a well-known dataset the off-diagonal structure is often interpretable — which two classes get confused tells you something about the representation, not just the score.

Four Counts, Per Class

For a class j, every prediction falls into one of four cells:

	truly in j	not in j
assigned to j	TP_j — true positives	FP_j — false positives
not assigned to j	FN_j — false negatives	TN_j — true negatives

> Read the letters as **decision then correctness**: P/N is what h decided, T/F is whether that decision was right.

Speaker notes

The mnemonic in the tip box removes nearly all of the confusion students have with these four terms. Have them classify a concrete example aloud before moving on.

Precision and Recall

Recall — completeness

$$\mathrm{rec}_j = \frac{TP_j}{TP_j + FN_j}$$

Of everything that truly belongs to j, what fraction did h find?

Precision — exactness

$$\mathrm{prec}_j = \frac{TP_j}{TP_j + FP_j}$$

When h says j, how often is it right?

Both are **conditional probabilities with different conditions** — recall conditions on the truth, precision on the prediction. Swapping them is the same error as swapping the conditionals in Bayes' theorem.

One number that is high only if both are: their harmonic mean, $F_1 = \frac{2 \cdot \mathrm{prec}_j \cdot \mathrm{rec}_j}{\mathrm{prec}_j + \mathrm{rec}_j}$.

Speaker notes

The link to Bayes is the payoff of teaching probability first: precision and recall are P(correct | predicted) and P(found | true). Ask which one a search engine optimises and which one a screening programme optimises.

Mini-Exercise

A spam filter flags 80 of 1,000 emails; 60 of them are spam, out of 100 spam emails in total. What are precision and recall — and which would you rather improve?



Speaker notes

Solution on the same slide, click-gated. The final sentence is the one to stabilise — the choice between precision and recall is a statement about costs, which is why the cost-matrix slide came earlier. If there is time, contrast with the summary numbers: F1 = 0.67, while accuracy is (60 + 880)/1000 = 0.94 — the number that looks best says least.

Mini-Exercise

A spam filter flags 80 of 1,000 emails; 60 of them are spam, out of 100 spam emails in total. What are precision and recall — and which would you rather improve?

- **Precision** $= 60/80 = 0.75$: a quarter of flagged mail is legitimate.
- **Recall** $= 60/100 = 0.60$: forty spam messages still arrive.

For a spam filter, **precision** usually matters more: a lost legitimate email costs far more than a spam message that slips through. The cost matrix decides, not the metric.

Takeaway

> The confusion matrix shows **which** errors occur; precision and recall split "being wrong" into **being imprecise** and **being incomplete**. Which one you optimise is a decision about costs, not about mathematics.

The Churn Shop: Accuracy or Recall?

Of **1,000** customers, **30** will churn in the next quarter (3%). Two models:

	flags	churners caught	accuracy	recall	precision
"nobody churns"	0	0 of 30	97.0%	0%	—
churn alert	60	24 of 30	95.8%	80%	40%

Speaker notes

The running example of the course, with the base rate of Lecture 02's trap. Recompute with the students: the alert makes 36 false alarms and misses 6 churners, so it gets 24 + 934 = 958 of 1,000 right; the constant model gets the 970 non-churners right. Recall 24/30, precision 24/60.

The last line previews the decision view: if a retention call costs little and a lost customer much, 36 unnecessary calls are cheap. Logistic regression and naive Bayes both produce scores; the threshold that turns a score into "call" is chosen by these costs, not by accuracy.

The Churn Shop: Accuracy or Recall?

Of **1,000** customers, **30** will churn in the next quarter (3%). Two models:

	flags	churners caught	accuracy	recall	precision
"nobody churns"	0	0 of 30	97.0%	0%	—
churn alert	60	24 of 30	95.8%	80%	40%

> The useless model wins on accuracy. At a 3% base rate, accuracy mostly counts the customers who were never at risk.

The Churn Shop: Accuracy or Recall?

Of **1,000** customers, **30** will churn in the next quarter (3%). Two models:

	flags	churners caught	accuracy	recall	precision
"nobody churns"	0	0 of 30	97.0%	0%	—
churn alert	60	24 of 30	95.8%	80%	40%

The useless model wins on accuracy. At a 3% base rate, accuracy mostly counts the customers who were never at risk.

Which model is better depends on the **costs**: a retention call for 60 customers against 24 kept.

Squared Error

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(h(\mathbf{x}_i) - y_i\right)^2 \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}$$

MSE

Average quadratic loss; always non-negative, lower is better.

Unit is the **square** of the target's unit — hard to interpret directly.

Large errors are penalised disproportionately.

RMSE

Same ordering as MSE, but back in the **target's own unit** — which is why it is the one usually reported.

Speaker notes

The quadratic penalty is a modelling choice with consequences: it makes the fit sensitive to outliers, which connects back to robustness in descriptive statistics. Ask when that sensitivity is desirable — when large errors really are disproportionately costly.

Absolute and Relative Error

MAE

$$\frac{1}{n}\sum_i |h(\mathbf{x}_i) - y_i|$$

Easier to interpret than RMSE; treats all errors proportionally.

Mean error

Drop the absolute value to keep the **direction** — reveals systematic over- or under-prediction; errors of opposite sign cancel.

Mean relative error

$$\frac{1}{n}\sum_i \frac{|h(\mathbf{x}_i) - y_i|}{|y_i|}$$

The average error *per unit of target* — comparable across scales, undefined where $y_i = 0$.

Normalised RMSE variants aim to make errors comparable between datasets, but are hard to interpret. Prefer a measure whose unit you can name.

Speaker notes

The mean error (with sign) is underused and worth highlighting: a model can have excellent RMSE and still be biased in one direction, which matters for forecasting and pricing.

Takeaway

Squared measures punish large errors and lose the unit; absolute measures stay interpretable; signed measures expose bias. Report the one whose **unit and failure mode** match the decision the prediction feeds.